Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

Факультет «Инженерный бизнес и менеджмент»

Кафедра «Промышленная логистика»

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Разработка пользовательских веб-интерфейсов»

Тема: «Построение пользовательских интерфейсов: HTML/CSS. Разработка калькулятора в стиле дизайн-системы сайта Яндекс».

Выполнила:
студентка группы ИБМ3-54Б
Горбунова В.С.

Москва, 2026 г.

1. Цель работы

Целью лабораторной работы №2 является изучение базовых концепций и инструментов языка сценариев JavaScript, принципов работы с объектной моделью документа DOM (Document Object Model), а также практическое освоение навыков обработки пользовательских событий (клик мыши). В ходе выполнения работы решена задача связывания ранее разработанного статичного HTML/CSS-интерфейса калькулятора с динамическими алгоритмами для выполнения математических расчетов в реальном времени.

2. Описание архитектуры файлов проекта

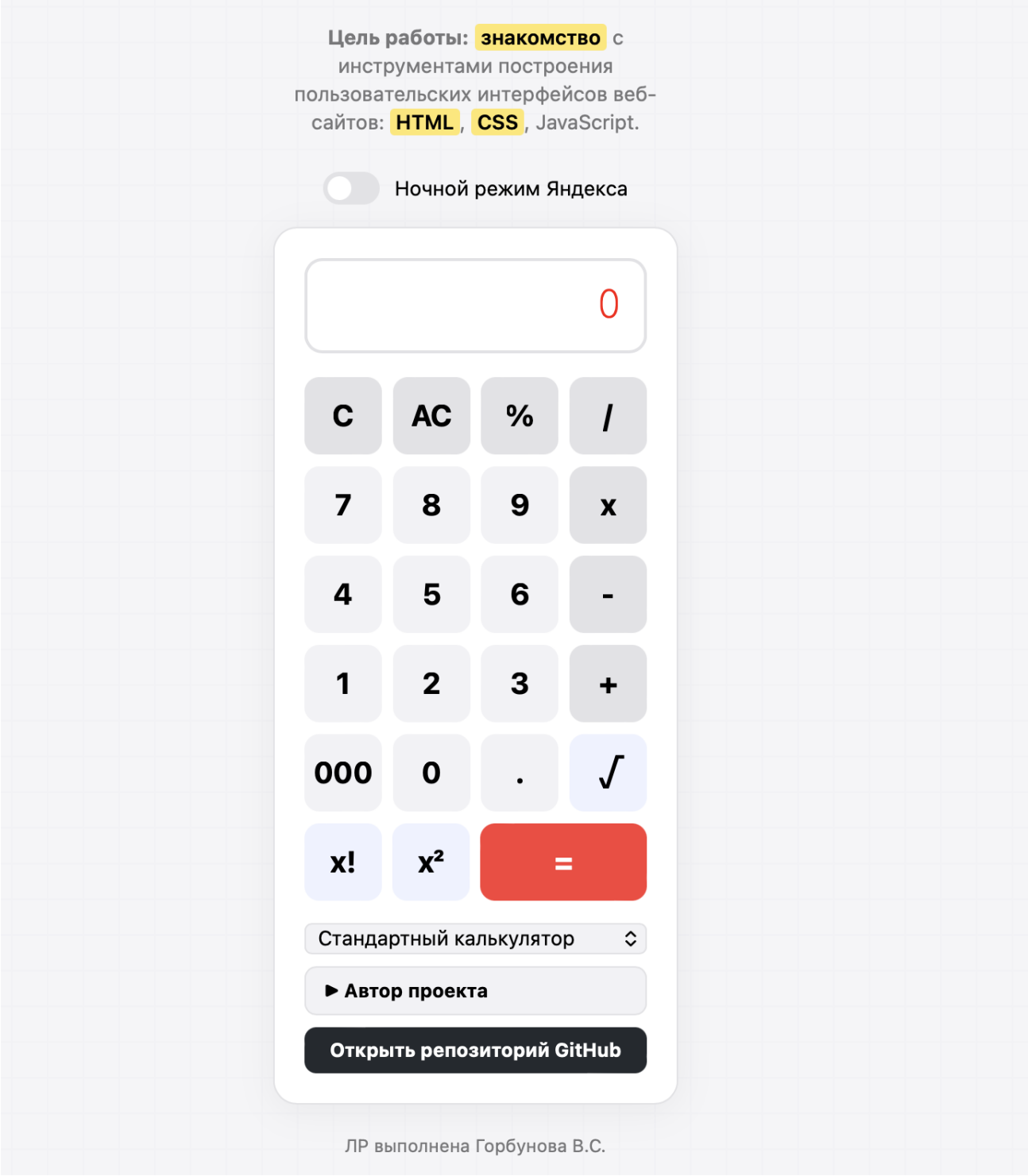
В соответствии с требованиями модульности и разделения зон ответственности (структура/стили/логика), архитектура интерактивного приложения была расширена до трех независимых файлов:

calculator.html — файл разметки, содержащий иерархический каркас калькулятора. Все активные элементы (кнопки, дисплей) снабжены уникальными атрибутами id для прямого доступа к ним из скрипта. Подключение JavaScript-логики вынесено в самый низ тела документа (перед закрывающим тегом </body>), что гарантирует построение DOM-дерева до начала выполнения сценариев.

style.css — таблица стилей, реализующая дизайн-систему Яндекса, сеточную адаптивную Flexbox-структуру и стили для новых инженерных кнопок.

script.js — исполняемый файл сценария на языке JavaScript. Содержит буферную оперативную память приложения, функции фильтрации пользовательского ввода, алгоритмы вычислений и обработчики событий.

3. Техническое описание реализации функций работы кнопок



Ниже приведено подробное описание алгоритмов работы стандартных и самостоятельно разработанных инженерных функций калькулятора, интегрированных в файл script.js:

А. Инициализация и оперативная память (Шаги 2, 3 методических указаний)

Вся логика заперта внутри глобального обработчика window.onload = function() { ... };. Это гарантирует, что JS-код начнет выполняться строго после того, как браузер полностью построит структуру страницы. Объявлена буферная «оперативная память» калькулятора, состоящая из четырех изменяемых переменных:

let a = ''; — строковый буфер для хранения первого числа (операнда).

let b = ''; — строковый буфер для хранения второго числа (операнда).

let selectedOperation = null; — буфер для хранения текущего знака бинарной операции (+, -, x, /).

let expressionResult = ''; — переменная промежуточного итога.

Доступ к дисплею реализован через системный метод document.getElementById("result"). Сбор всех цифровых кнопок в единую коллекцию выполнен через современный селектор атрибутов document.querySelectorAll('[id^="btn_digit_"]'). Обход коллекции и подписка на клики button.onclickосуществлены при помощи встроенного итератора .forEach().

Б. Базовые алгоритмы ввода и арифметических вычислений

Ввод чисел и конкатенация: Функция onDigitButtonClicked(digit) считывает текстовое значение нажатой кнопки (innerHTML). Если знак операции еще не выбран (selectedOperation === null), вводимые цифры посимвольно склеиваются (конкатенируются) в строку a. Если операция выбрана — цифры склеиваются в строку b.

Контроль ввода точки: Интегрирована проверка if (!a.includes('.')). Если в буфере уже присутствует десятичный разделитель, повторный ввод точки

блокируется, что предотвращает появление математически некорректных чисел вида 5.6.7.

Выполнение бинарных расчетов (calculateResult): При клике на кнопку равенства (=) или при вводе цепочки непрерывных вычислений (например, 2 + 3 + 4...) запускается функция calculateResult(). На основе оператора множественного выбора switch(selectedOperation) строки преобразуются в математические числа функцией parseFloat(), после чего выполняется нужное арифметическое действие. По окончании расчета буфер b и знак операции очищаются, а результат записывается в буфер a и выводится на дисплей.

Защита от деления на ноль: Внутри блока case '/': реализована проверка numB === 0. При попытке деления на ноль калькулятор блокирует стандартную ошибку браузера (Infinity) и выводит на дисплей понятное пользователю сообщение "Ошибка", полностью очищая буферы памяти.

4. Реализация индивидуальных инженерных заданий

1. Кнопка полной очистки памяти «AC» (All Clear)

В отличие от стандартной очистки, данная функция полностью обнуляет все регистры калькулятора.

Реализация: Кнопке с id="btn_op_clear_all" присвоен обработчик, который принудительно очищает операнды a = '', b = '', сбрасывает оператор selectedOperation = null, обнуляет итог и принудительно выводит на экран стартовый ноль '0'. Это позволяет пользователю мгновенно прервать любую цепочку вычислений.

2. Кнопка посимвольного удаления «C» (Умный Backspace)

Реализует функцию стирания последней ошибочно введенной цифры.

Реализация: Метод считывает, какой операнд сейчас заполняется. С помощью встроенного метода обработки строк .slice(0, -1) от текущей строки

отрезается один крайний правый символ. Если в строке оставался последний символ, после его удаления буферу присваивается значение '0'.

3. Ввод фиксированных трех нулей «000»

Специализированная клавиша для быстрого ввода крупных сумм.

Реализация: Внутри функции ввода прописано условие if (digit === '000'). Программа проверяет: если буфер числа пуст или равен '0', ввод трех нулей игнорируется (чтобы избежать появления некорректных чисел вроде 00005). Кнопка срабатывает и конкатенирует += '000' только если в операнде уже есть хотя бы одна значащая цифра от 1 до 9.

4. Операция вычисления процента «%»

Реализация: Функция определяет, какое число сейчас находится на экране. Строка конвертируется в число через parseFloat(), делится на 100, переводится обратно в строковый формат .toString() и выводится на дисплей, позволяя мгновенно находить сотые доли числа.

5. Извлечение квадратного корня «√»

Реализация: Функция обращается к встроенной математической библиотеке языка JavaScript — Math.sqrt(val). Перед расчетом выполняется жесткая проверка: если пользователь пытается извлечь корень из отрицательного числа, алгоритм выдает сообщение "Ошибка" и сбрасывает память. В случае валидного числа результат записывается в текущий операнд.

6. Возведение числа в квадрат «x^2»

Реализация: Математическая функция унарного типа. Программа считывает активное число, преобразует его в числовой тип и перемножает само на себя: parseFloat(a) * parseFloat(a). Это обеспечивает мгновенное возведение в степень без задействования второго буфера.

7. Итерационное вычисление факториала «x!»

Реализация: Для расчета факториала числа (\(N!\)) написан классический цикл for. Скрипт принимает целое число через функцию parseInt(). Запускается

цикл for (let i = 1; i <= val; i++) { fact *= i; }, который последовательно перемножает все натуральные числа.

Защита от зависания: В коде установлена жесткая защита if (val > 13). Поскольку факториал растет лавинообразно, вычисление значений выше 13 приводит к переполнению разрядной сетки и выдаче системной ошибки. При превышении этого лимита калькулятор безопасно выводит сообщение "Слишком много".

8. Жесткое ограничение разрядности дисплея (До 10 цифр)

Реализация: Во избежание поломки интерфейса, в алгоритм ввода внедрена регулярная проверка .replace('.', '').length >= 10. Если пользователь пытается вручную набрать число, содержащее более 10 значащих цифр, скрипт блокирует выполнение операции return, не позволяя тексту растягивать карточку.

9. Интеллектуальное динамическое округление дробей

Реализация: Главная инженерная доработка. Написана универсальная функция formatOutputValue(value). При выводе сложных бесконечных дробей (например, 1 / 3 = 0.3333...) программа вычисляет длину целой части числа. Затем из жесткого лимита в 10 знаков вычитается это пространство, и метод .toFixed(availableDigitsForFraction) округляет дробный хвост ровно до того количества знаков, сколько свободных мест осталось в окошке. Повторный вызов parseFloat() аккуратно отсекает лишние системные нули на конце.

10. Автомасштабирование шрифта (Автоподбор размера без троеточий)

Реализация: Написана функция adjustFontSize(text). Она проверяет финальную длину строки, подготовленной к выводу на экран. Если длина строки превышает 8, 11 или 14 символов (с учетом знаков минуса или точек), функция динамически уменьшает CSS-свойство outputElement.style.fontSize (с исходных 1.9rem до 1.6rem, 1.35rem или 1.1rem). В сочетании с добавленным в CSS свойством плавности transition: font-size 0.15s ease;, цифры при заполнении

экрана автоматически и плавно уменьшаются в размерах, гарантируя полное отсутствие текстовых троеточий и скрытых символов.

5. Заключение

В ходе выполнения лабораторной работы №2 были успешно освоены практические навыки программирования динамического поведения веб-ресурсов на языке JavaScript. В полном объеме изучены методы манипуляции элементами DOM-дерева по их уникальным идентификаторам id, принципы перехвата и обработки событий клика мыши, а также математические методы округления и преобразования типов данных (parseFloat, toPrecision, toFixed). Разработанный калькулятор успешно сочетает в себе визуальную минималистичную эстетику дизайн-системы Яндекса и продвинутую, отказоустойчивую инженерную логику расчетов.